Colégio Técnico da UFMG

Relatório As-Built

Sistema de Controle de Tráfego Aéreo e Pistas de Pouso

Nome: João Pedro Vidal Alves
Curso: *Desenvolvimento de Sistemas Subsequente*
Período: 2º Semestre

27 de setembro de 2026

1 Mapeamento de Requisitos

Proposição da solução

- O trabalho propõe um protótipo de Controlador Aéreo Nacional dividido em três componentes independentes, cada um mapeado para a estrutura de dados que melhor reflete sua regra de negócio: fila de prioridade para a autorização de pouso, lista circular duplamente encadeada para a circulação em taxiways e uma combinação de lista + pilha para o despacho de bagagem.
- Optou-se por implementar apenas os requisitos obrigatórios do enunciado, deixando de fora explicitamente as políticas e mecanismos citados como opcionais ("podem ser elaboradas", "caso tenha interesse"). Essas alternativas estão listadas na Seção 2.1.3 (e equivalentes) de cada componente, como sugestões de evolução futura.

Divisão dos códigos

- Existem soluções conjuntas: os **Componentes 1 e 2 foram implementados juntos**, em um único arquivo, porque a autorização de pouso depende diretamente do estado do circuito de taxiway — uma aeronave só sai da fila de prioridade quando há espaço físico confirmado para ela circular em solo.
- Foram entregues **2 códigos-fonte em C**: um cobrindo os Componentes 1+2 (integrados) e outro cobrindo o Componente 3, isolado.
- A razão da combinação 1+2 é a dependência funcional direta entre eles (ver acima). O Componente 3, por outro lado, opera sobre estruturas totalmente próprias (manifesto e porão) e não precisa reaproveitar nenhum tipo ou função dos outros dois — por isso foi mantido em arquivo separado, podendo ser testado de forma independente.

Identificação dos códigos

A tabela abaixo identifica cada arquivo entregue, o(s) componente(s) que ele resolve e seus pré-requisitos:

Arquivo	Componente(s) implementados	Pré-requisitos / bibliotecas
`pouso_taxiway.c`	Componente 1 (fila de prioridade de pouso) + Componente 2 (circuito de taxiway), integrados	Apenas biblioteca padrão do C (stdio.h, stdlib.h, string.h). Compila com gcc padrão, sem flags especiais.
`bagagem.c`	Componente 3 (manifesto + porão de bagagem), independente	Apenas biblioteca padrão do C (stdio.h, stdlib.h, string.h). Compila com gcc padrão, sem flags especiais.

2 Componente 1

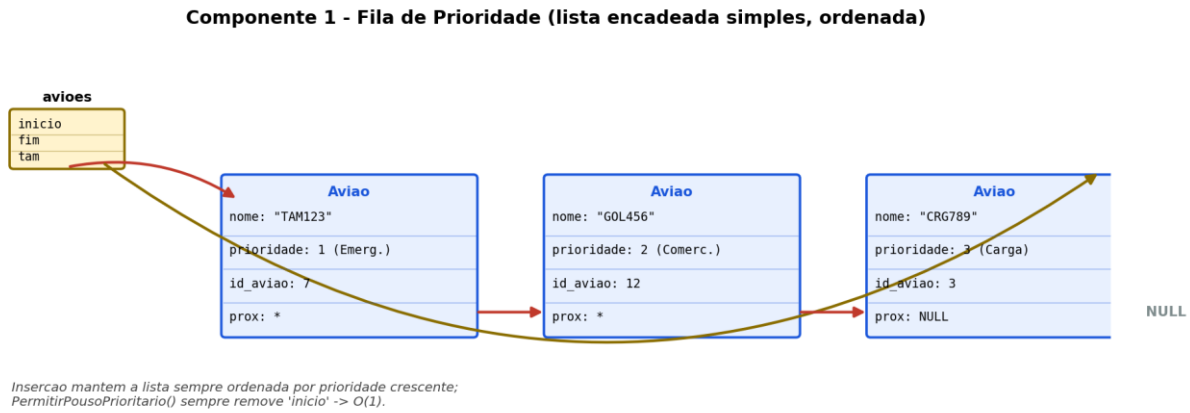
Descrição da solução, das funções e demais propostas de solução para o **Aproximação e Autorização de Pouso**.

2.1 Arquitetura

Proposta da solução e da rotina/algoritmo focado no problema de **Aproximação e Autorização de Pouso**.

A regra operacional exige que uma aeronave de nível de prioridade numericamente inferior (Emergência = 1, Comercial = 2, Carga = 3) seja sempre autorizada a pousar antes de uma de nível superior, independentemente da ordem de chegada — o que caracteriza uma fila de prioridade. A autorização em si foi implementada de forma integrada ao Componente 2: o avião no início da fila é apenas consultado (sem ser removido) até que exista um nó de taxiway livre; só então ele sai da fila e o pouso é de fato autorizado.

2.1.1 Diagramação



Estado de memória com três aeronaves aguardando pouso, já ordenadas por prioridade crescente. As setas vermelhas curvas partem da estrutura de controle (inicio/fim) para o primeiro e o último nó; as setas retas representam o campo prox de cada nó.

2.1.2 Detalhamento do Algoritmo e Estrutura de Dados

Tipo de arquitetura de dados proposta

- Lista encadeada simples, mantida sempre ordenada por prioridade crescente no momento da inserção (não é um vetor nem uma fila circular).

Tipo e formato da(s) variável(is)

- `enum prioridade` (Emergencia=1, Comercial=2, Carga=3);
- strings fixas (`char nome[20]`) e inteiros (`int id_aviao`) nos nós; ponteiros (`AviaoFila*`) para encadeamento.

Descrição dos dados alocados

- `AviaoFila`: nome, prioridade, id_aviao, ponteiro `prox`.
- `FilaPouso`: ponteiros `inicio` e `fim`, contador `tam`.

Benefícios observados por essa estrutura

- Autorização de pouso (operação mais frequente e mais crítica em tempo) é O(1): basta remover o nó `inicio`.

- Sem limite fixo de aeronaves em espera e sem realocação de memória, ao contrário de um vetor dinâmico.
- A inserção ordenada preserva a ordem de chegada dentro do mesmo nível de prioridade (estabilidade).

Possíveis pontos de problema de escalabilidade

- A inserção é O(n) no pior caso, pois percorre a lista até achar a posição correta; em um aeroporto com centenas de aeronaves simultâneas na fila, isso degrada linearmente. Uma fila de prioridade baseada em heap binário ofereceria inserção O(log n) mantendo a remoção do topo eficiente.
- O sistema é single-thread e não trata concorrência: se o protótipo evoluísse para múltiplas torres/operadores atuando ao mesmo tempo sobre a mesma fila, seria necessário adicionar sincronização (mutex/semáforos) para evitar condições de corrida.

2.1.3 Regras de Negócio

Implementações que não foram executadas nesta entrega, mas que poderiam ser consideradas em uma solução real:

- Geração automática de solicitações de pouso para testes de carga (em vez de entrada manual pelo operador).
- Reavaliação dinâmica de prioridade em tempo real (ex.: elevar a prioridade de uma aeronave comercial que está ficando com combustível crítico, promovendo-a a Emergência).
- Suporte a múltiplos operadores/torres atuando simultaneamente sobre a mesma fila (exigiria controle de concorrência).

3 Componente 2

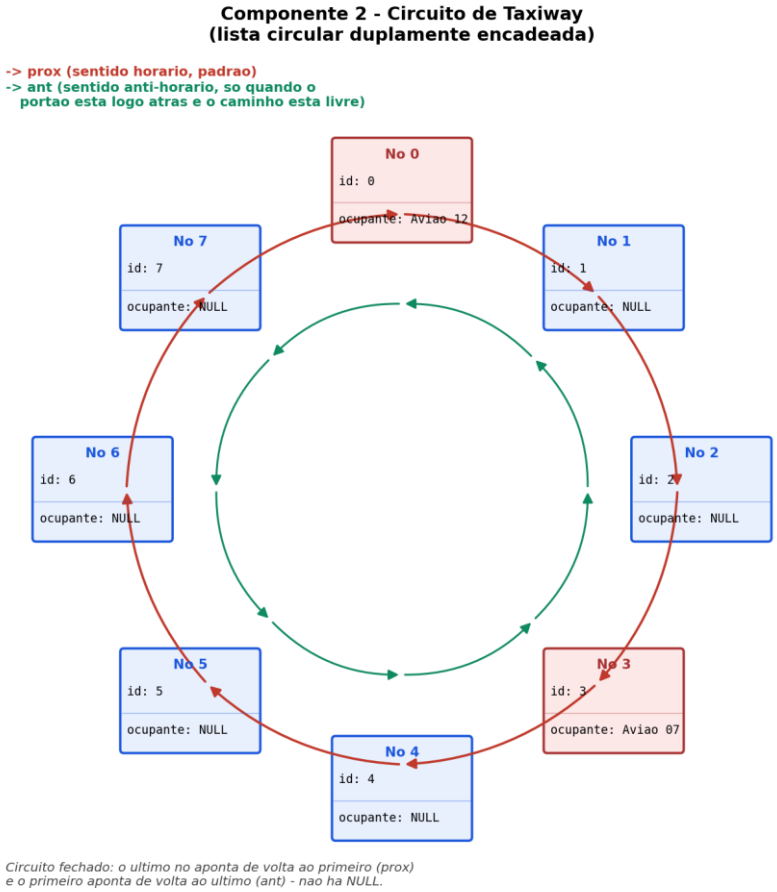
Descrição da solução, das funções e demais propostas de solução para o **Circulação e Gerenciamento de Solo (Taxiways)**.

3.1 Arquitetura

Proposta da solução e da rotina/algoritmo focado no problema de **Circulação e Gerenciamento de Solo (Taxiways)**.

O enunciado descreve as taxiways como uma rotatória: não há início nem fim físico, e a aeronave circula continuamente até liberar o portão — o que mapeia diretamente para uma lista circular. Como o sistema precisa navegar em sentido horário (padrão) e, em certas condições, em sentido anti-horário (quando o portão está logo atrás e o caminho está livre), a lista precisa ser duplamente encadeada.

3.1.1 Diagramação



Circuito com 8 nós. Nós em vermelho estão ocupados por uma aeronave (campo ocupante preenchido). As setas externas (vermelhas) representam o encadeamento prox em sentido horário; as internas (verdes) representam ant, em sentido anti-horário. Por ser circular, não existe ponteiro NULL.

3.1.2 Detalhamento do Algoritmo e Estrutura de Dados

Tipo de arquitetura de dados proposta

- Lista circular duplamente encadeada (ponteiros `prox` e `ant`), com número fixo de nós definido na inicialização.

Tipo e formato da(s) variável(is)

- Inteiros para identificadores de nó e de aeronave; ponteiros (`NoTaxiway*`, `AviaoTaxi*`) para encadeamento e ocupação.

Descrição dos dados alocados

- `NoTaxiway`: id, ponteiro `ocupante` (ou NULL), ponteiros `prox`/`ant`.
- `AviaoTaxi`: nome, id_aviao, portao_destino, ponteiro para o nó onde está.
- `Circuito`: ponteiro de referência qualquer do círculo e a lista de aviões em solo.

Benefícios observados por essa estrutura

- Entrada e saída de uma aeronave do circuito é O(1) e não requer inserir/remover nós: só o campo `ocupante` muda, então a circulação das demais nunca é interrompida.
- A lista circular elimina o caso especial de "fim de lista": o próximo nó sempre existe.
- O ponteiro duplo permite decidir o sentido de deslocamento em O(1) por passo.

Possíveis pontos de problema de escalabilidade

- O número de nós é fixo desde a criação do circuito; não é possível expandir ou reduzir a infraestrutura em tempo de execução (uma ampliação real do aeroporto exigiria reiniciar o sistema com um novo valor de quantidade_nos).
- O cálculo de distância e verificação de caminho livre percorrem a lista nó a nó (O(distância) por aeronave por tick); em um circuito muito grande com muitas aeronaves simultâneas, o custo por rodada de simulação cresce proporcionalmente ao produto dos dois.
- A resolução de impasse por espera simples não trata o caso patológico de bloqueio mútuo indefinido (duas aeronaves impedindo-se mutuamente sem nunca avançar); não há timeout nem critério de desempate por prioridade.

3.1.3 Regras de Negócio

Implementações que não foram executadas nesta entrega, mas que poderiam ser consideradas em uma solução real:

- Pouso forçado de emergência mesmo sem taxiway livre, bloqueando temporariamente a pista para novos pousos.
- Políticas alternativas de contingência quando o circuito está lotado: órbita de espera, controle de fluxo em rota, gerenciamento extra de pátio/portões, desvio para outro aeroporto, restrição de decolagem na origem.
- Políticas extras de taxiway citadas no enunciado como opcionais: distância mínima de segurança entre aeronaves e capacidade máxima por trecho.
- Temporizador real simulando o tempo físico de deslocamento, em vez do avanço manual por passo ("tick") usado nesta entrega.

4 Componente 3

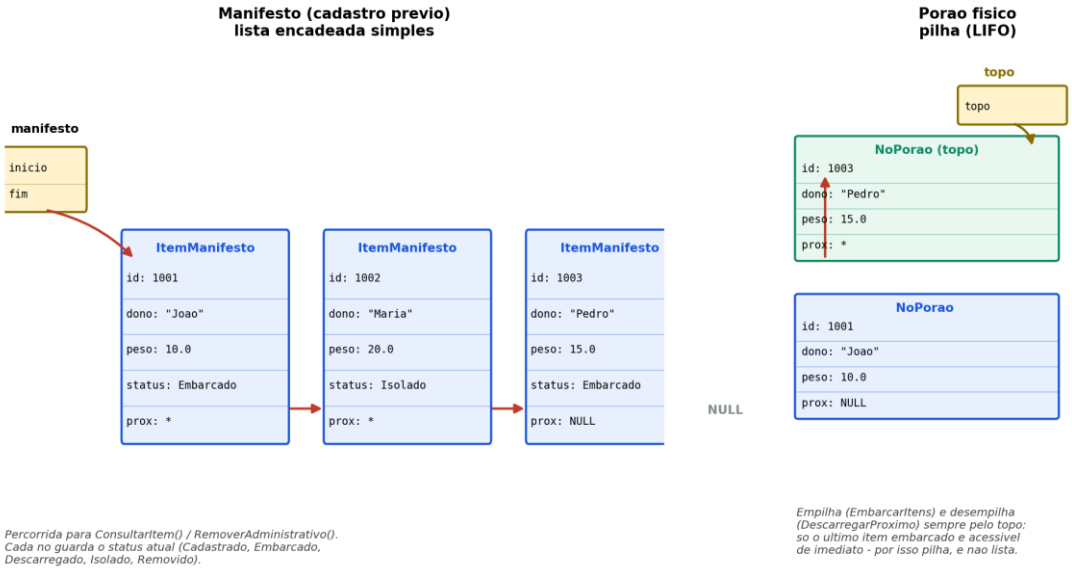
Descrição da solução, das funções e demais propostas de solução para o **Despacho de Porão, Cadastro Prévio e Inspeção**.

4.1 Arquitetura

Proposta da solução e da rotina/algoritmo focado no problema de **Despacho de Porão, Cadastro Prévio e Inspeção**.

Este componente reúne dois sub-problemas de naturezas diferentes, tratados com duas estruturas distintas. O cadastro prévio (manifesto) precisa de consulta arbitrária por id e remoção administrativa antes do embarque — operações típicas de uma lista encadeada simples. Já o porão físico é explicitamente restrito pelo enunciado ("o operador só tem acesso imediato ao último contêiner ou bagagem embarcado"), o que é, por definição, uma disciplina LIFO — uma pilha. Por decisão de projeto, este componente foi mantido isolado dos demais, em arquivo próprio, sem nenhuma referência aos Componentes 1 ou 2.

4.1.1 Diagramação



À esquerda, o manifesto como lista encadeada simples, com o item 1002 já marcado como Isolado pela inspeção de emergência. À direita, o estado do porão físico como pilha após a mesma inspeção: o item 1002 foi removido do meio da estrutura e a ordem relativa dos itens 1003 (topo) e 1001 foi preservada, restaurada pelo algoritmo de pilha auxiliar.

4.1.2 Detalhamento do Algoritmo e Estrutura de Dados

Tipo de arquitetura de dados proposta

- Lista encadeada simples (manifesto) + pilha/lista encadeada em LIFO (porão físico), estruturas independentes ligadas apenas pelo id do item.

Tipo e formato da(s) variável(is)

- `enum StatusItem` (Cadastrado, Embarcado, Descarregado, Isolado);
- inteiros e strings (`int id`, `char dono[30]`, `float peso`) nos nós; ponteiros (`ItemManifesto*`, `NoPorao*`) para encadeamento.

Descrição dos dados alocados

- `ItemManifesto` (lista): id, dono, peso, status, prox.

- `NoPorao` (pilha): id, dono, peso, `prox` (aponta para o item imediatamente abaixo).
- `SistemaBagagem`: ponteiros de início/fim do manifesto e para o `topo` da pilha.

Benefícios observados por essa estrutura

- Empilhar (embarque) e desempilhar (descarregamento com double-check) são O(1), refletindo fielmente a restrição mecânica real do porão.
- O manifesto, por ser independente da pilha, permite consultar ou remover administrativamente qualquer item por id sem tocar na estrutura física.
- A interceptação de emergência usa uma pilha auxiliar: duas transferências entre pilhas invertem a ordem duas vezes, restaurando a ordem original dos itens remanescentes automaticamente.

Possíveis pontos de problema de escalabilidade

- Consulta e remoção no manifesto são O(n), pois a lista é percorrida sequencialmente sem nenhum índice auxiliar; uma tabela hash por id tornaria essas operações O(1) amortizado para manifestos muito grandes.
- A inspeção de emergência é O(n) no pior caso, pois pode ser necessário desempilhar todo o porão até encontrar o item alvo — relevante em porões com grande volume de bagagens.
- Por ser um único manifesto/porão por execução, o programa não escala para múltiplas aeronaves simultâneas sem duplicar/reiniciar a estrutura; uma versão com registros indexados por id_aviao (testada em uma iteração anterior deste projeto) resolveria isso, mas foi descartada nesta entrega para manter o componente isolado dos demais.

4.1.3 Regras de Negócio

Implementações que não foram executadas nesta entrega, mas que poderiam ser consideradas em uma solução real:

- Integração automática com o Componente 2, liberando o descarregamento assim que a aeronave estaciona no portão (nesta entrega o componente é isolado e não recebe esse evento).
- Suporte a múltiplas aeronaves simultâneas, cada uma com seu próprio manifesto e porão, indexados por id_aviao.
- Persistência em arquivo ou banco de dados — atualmente todo o estado existe apenas em memória e é perdido ao encerrar o programa.
- Validações adicionais no double-check, como código de barras do item ou dimensões físicas, além de dono e peso.